

Rendus soutenance 2:

26/03/2026

WHAT'S NEXT?

**Hadrien Beaurepaire
Ghali Benani
Nael Belhadj Amara
Julia Guimaraes
Melissa Gay**

Plan soutenance:

- 1) Presentation bref du projet: 30 s

Nael

- 2) Présentation du plan de la soutenance: 30s

Ghali

- 3) Récapitulatif des avancés: 30s

Hadrien

- 4) Feedback de la dernière soutenance: 30s

Melissa

- 5) Attendues deuxième soutenance: 30s

Ghali

- 6) Analyse des difficultés: 30 s

Julia

- 7) Nouvelles avancés: 9 min

Julia - Ghali - Melissa - Hadrien - Nael

- 8) Conclusion: 30 s

Hadrien

Récapitulatif de nos avancées avant la première soutenance:

Introduction:

Dans le cadre de notre projet de développement de jeux vidéo intitulé “What’s Next ?”, nous avons travaillé en groupe afin de créer un jeu basé sur des énigmes et la coopération entre deux joueurs. Notre groupe était composé de cinq personnes : Hadrien Beaurepaire, Ghali Bennani, Nael Belhadj Amara, Julia Guimaraes et Melissa Gay.

Notre objectif était de créer un jeu en 2D dans lequel deux joueurs doivent coopérer pour résoudre des puzzles et progresser dans différentes salles.

Organisation du projet:

Au début du projet, nous avons commencé par remplir le cahier des charges afin de définir les objectifs du jeu et les différentes tâches à réaliser.

Nous avons choisi de faire un jeu basé sur des puzzles. Les joueurs doivent s’entraider pour résoudre des énigmes et avancer dans les différentes salles.

Nous avons également choisi de faire un jeu en 2D, car cela correspondait mieux à notre niveau en programmation de jeux vidéo. En plus de la programmation, nous avons décidé de créer nous-mêmes les graphismes, les sons et le site web afin d’avoir un projet plus personnalisé.

Pour la répartition du travail, nous avons essayé de donner les tâches en fonction des compétences de chacun. Au début, nous voulions travailler principalement tous ensemble. Mais nous avons vite remarqué que ce n’était pas toujours possible à cause des emplois du temps différents. Nous avons donc parfois travaillé seuls ou en petits groupes, puis partagé nos avancées avec les autres.

Outils de communication et de travail:

Pour communiquer entre nous, nous avons d’abord créé un groupe Snapchat. Cependant, nous avons rapidement vu que ce n’était pas très pratique pour partager des documents. Nous avons donc décidé de passer sur WhatsApp, qui permet de mieux envoyer des fichiers. Ensuite, nous avons créé un serveur Discord afin de mieux organiser les discussions. Cela nous permet d’avoir plusieurs canaux pour les différents sujets du projet.

Pour le développement du jeu, nous avons utilisé Visual Studio Code pour programmer et Git pour gérer le code. Nous avons également utilisé GitHub pour partager le projet et garder une sauvegarde du code.

Le scénario du jeu:

Nous avons complètement fini le script principal du jeu et nous avons désormais créé une charte qui détaille le script.

Création du site web:

Nous avons également créé un site web pour présenter notre projet. Ce site a été développé par Mélissa.

Le site contient plusieurs pages :

- une page Accueil qui présente rapidement le jeu
- une page Lore qui explique l'histoire du jeu
- une page Groupe qui présente les membres de l'équipe
- une page Liens qui donne accès aux différents documents du projet

Développement du menu du jeu:

Le menu du jeu a été créé par Nael en utilisant Python et la bibliothèque Pygame.

Au début, le menu était très simple. Ensuite, plusieurs améliorations ont été ajoutées :

- un menu Play avec les options "Nouvelle partie", "Continuer" et "Retour"
- un menu Paramètres avec un réglage de luminosité
- un mode plein écran pour améliorer l'affichage
- des effets sonores lors des clics sur les boutons
- des effets visuels lorsque la souris passe sur un bouton
- un fond animé en vidéo pour rendre le menu plus immersif

Développement des salles:

Le développement des salles a été principalement réalisé par Julia.

Elle a commencé par apprendre les bases de Pygame, notamment les fonctions permettant d'afficher des images, dessiner des formes et gérer les mouvements des personnages.

La première salle a été conçue pour apprendre aux joueurs les bases du jeu. Elle repose sur la coopération entre les deux personnages.

La deuxième salle repose sur une énigme logique. Dans cette salle, il y a quatre statues avec des symboles : le Soleil, la Lune, la Racine et le Cristal. Chaque joueur voit des indices différents sur les murs. Ils doivent donc communiquer entre eux pour trouver l'ordre correct dans lequel activer les statues. Si les joueurs se trompent, la salle se réinitialise. Lorsqu'ils trouvent la bonne solution, la porte s'ouvre et ils peuvent passer à la suite.

Graphismes:

Les graphismes du jeu ont été créés par Hadrien à l'aide de l'intelligence artificielle et d'une tablette graphique.

Les personnages principaux ont d'abord été générés par IA, puis modifiés afin de créer plusieurs sprites pour les animations. Cela permet d'avoir des mouvements plus fluides lorsque les personnages se déplacent.

Hadrien a également créé des packs de textures pour les murs, les sols et les objets interactifs.

Pour créer les niveaux du jeu, Hadrien a utilisé le logiciel Tiled. Ce logiciel permet de construire facilement des maps en utilisant les textures. Les maps sont ensuite importées dans le jeu afin que les personnages puissent interagir avec les différents éléments.

Conclusion:

Nous avons déjà défini le scénario du jeu, créé le site web, développé un menu fonctionnel et commencé la programmation des premières salles. Les graphismes et les textures étaient également en cours de création.

Pour la suite du projet, nous devons continuer à développer les différentes salles du jeu, corriger certains problèmes techniques et améliorer les graphismes afin d'obtenir un jeu plus complet.

Feedback de la dernière soutenance et impacts sur notre travail

Lors de la dernière soutenance, nous avons reçu plusieurs retours qui nous ont permis de mieux comprendre la manière dont nous devons présenter notre projet et organiser notre travail.

Tout d'abord, nous avons réalisé que nous étions trop attachés aux exemples de déroulement de soutenance qui nous avaient été présentés. Nous avons essayé de reproduire ces exemples de manière assez stricte, sans forcément comprendre l'objectif réel de l'exercice.

Grâce aux retours reçus, nous avons compris que le but principal d'une soutenance est avant tout de présenter l'avancement réel du projet, d'expliquer les choix réalisés et de montrer les difficultés rencontrées. Il n'existe donc pas une seule manière correcte de présenter une soutenance, tant que nous arrivons à expliquer clairement notre progression et notre réflexion.

Nous avons également compris que nous n'étions pas aussi perdus que nous le pensions au début du projet. Certaines erreurs ou hésitations que nous avons rencontrées font en réalité partie du processus normal de développement d'un projet. Ces difficultés sont naturelles et permettent souvent de mieux comprendre les enjeux et les contraintes du projet.

Enfin, un point important qui est ressorti du feedback concerne l'organisation du travail au sein du groupe. Nous avons compris que l'organisation et la communication entre les

membres de l'équipe sont des éléments essentiels pour assurer une progression efficace du projet.

Suite à ces retours, nous avons essayé d'apporter quelques améliorations dans notre manière de travailler. Nous avons par exemple commencé à mieux définir certaines tâches et à clarifier les rôles de chacun dans le projet. Nous avons également essayé de documenter davantage certaines parties du jeu, notamment à travers des descriptions ou des chartes pour mieux partager la vision globale du projet.

Même si l'organisation reste encore un point que nous devons continuer à améliorer, ces retours nous ont permis d'identifier plus clairement ce problème et de réfléchir à des solutions pour mieux structurer notre travail pour la suite du développement du jeu.

Attendus deuxième soutenance

Script et lore:

Pour la deuxième soutenance, le script du jeu devait être finalisé à 100 %. Cela signifiait que l'histoire principale du jeu devait être complètement définie et cohérente avec le reste du projet.

Dans notre projet What's Next ?, le lore avait déjà été commencé lors de la première soutenance. Ghali avait proposé plusieurs idées d'histoire afin que le groupe puisse choisir celle qui correspondait le mieux au type de jeu que nous voulions créer. La version finale du lore a ensuite été retravaillée et rédigée par Nael afin de donner une histoire plus claire et plus immersive.

Pour cette soutenance, il était attendu que le lore et le thème des salles soient également finalisés à 100 %. Cela signifie que chaque salle devait avoir une idée claire, une logique dans le scénario et un lien avec l'univers du jeu. Le but était d'assurer une cohérence entre l'histoire, les puzzles et l'ambiance générale du jeu.

Menu du jeu:

Le menu du jeu devait être terminé à 100 % pour la deuxième soutenance. Le menu représente une partie importante de l'expérience utilisateur, car c'est la première chose que le joueur voit lorsqu'il lance le jeu.

Lors de la première soutenance, Nael avait déjà commencé à développer le menu. Pour la deuxième soutenance, l'objectif était d'avoir un menu complet et fonctionnel.

Développement des salles:

Le développement des salles devait atteindre environ 60 % d'avancement pour cette deuxième soutenance.

Les salles représentent les niveaux du jeu dans lesquels les joueurs doivent résoudre des énigmes pour progresser. Dans notre projet, plusieurs salles ont été imaginées avec des puzzles différents qui reposent principalement sur la coopération entre les deux joueurs.

Pour cette soutenance, l'objectif était donc d'avoir plusieurs salles déjà fonctionnelles afin de montrer que le gameplay du jeu fonctionne correctement.

Soundtrack:

La soundtrack du jeu devait également être finalisée à 100 % pour la deuxième soutenance.

Dans notre projet, les sons devaient être intégrés dans le menu et dans certaines interactions du jeu. L'objectif était d'avoir une ambiance sonore cohérente avec le thème du jeu et avec l'atmosphère des différentes salles.

IA et réseau:

Un autre élément attendu pour la deuxième soutenance concernait la mise en place des systèmes d'intelligence artificielle et de réseau, qui devaient être avancés à 100 %.

Level design:

Le level design devait être terminé à 100 % pour cette soutenance.

Le level design correspond à la conception des différentes salles du jeu : leur structure, la disposition des obstacles et la manière dont les puzzles sont organisés.

Dans notre projet, cela signifiait que toutes les salles prévues devaient être conceptualisées, même si certaines n'étaient pas encore complètement codées. L'objectif était d'avoir une progression logique dans la difficulté des puzzles et dans la manière dont les joueurs découvrent les mécaniques du jeu.

Game design

Enfin, le game design devait également être finalisé à 100 %.

Le game design correspond à l'ensemble des règles et des mécaniques du jeu. Cela inclut par exemple la manière dont les joueurs interagissent avec les objets, comment les puzzles fonctionnent ou encore comment la progression dans le jeu est organisée.

Dans What's Next ?, le game design repose principalement sur la coopération entre les deux joueurs. Les puzzles ont été conçus pour obliger les joueurs à communiquer et à travailler ensemble afin de progresser dans les différentes salles.

Conclusion:

La deuxième soutenance avait pour objectif de montrer que la majorité des éléments de conception du projet étaient finalisés. Le script, le lore, le menu, la soundtrack, le game design et le level design devaient être terminés, tandis que le développement des salles devait déjà être bien avancé.

Cette étape permettait de vérifier que le projet avançait correctement et que toutes les bases nécessaires étaient en place pour continuer le développement du jeu et préparer la soutenance finale.

Analyse des difficultés rencontrées lors de la réalisation des attendus de la deuxième soutenance

Lors de la préparation de la deuxième soutenance, nous avons rencontré plusieurs difficultés qui ont ralenti la réalisation de certains objectifs. Ces problèmes sont principalement liés à l'organisation du travail, à l'estimation du temps nécessaire pour certaines tâches et à une mauvaise anticipation de certaines parties du projet.

Cependant, ces difficultés nous ont permis de mieux comprendre la manière dont un projet de développement de jeu vidéo évolue réellement et nous ont aidés à avancer avec une vision plus claire de ce qui est réalisable.

Difficulté à mesurer l'avancement des salles:

L'un des premiers problèmes concerne l'objectif d'avoir 60 % des salles réalisées pour la deuxième soutenance.

Cet objectif s'est révélé difficile à évaluer car nous n'avions pas encore défini précisément combien de salles composeront la version finale du jeu. Sans connaître le nombre total de salles, il est compliqué de mesurer ce que représente réellement 60 % de progression.

Cela a rendu la planification du travail plus complexe et nous a montré l'importance de définir plus précisément certains objectifs dès le début du projet.

Difficulté à estimer le temps de travail:

Un autre problème important a été la difficulté à estimer le temps nécessaire pour réaliser certaines tâches.

Dans un projet de programmation, certaines fonctionnalités peuvent sembler simples à implémenter au départ, mais s'avèrent beaucoup plus complexes une fois que le travail commence réellement. À l'inverse, certaines idées peuvent sembler difficiles mais finalement être plus rapides à développer.

Comme il s'agit pour nous d'un premier projet de ce type, nous ne disposions pas encore de l'expérience nécessaire pour anticiper correctement la durée réelle de chaque tâche.

Cela a conduit notamment à une difficulté à estimer nos attentes en termes de nombres de salles, de difficulté des salles, de graphismes...

Organisation du travail en groupe:

L'organisation du travail à cinq personnes sur une période relativement courte a également été une difficulté importante.

Entre les cours, les travaux pratiques, les partiels et les rattrapages, il a été très difficile de trouver des moments où l'ensemble du groupe pouvait se réunir pour travailler ensemble. Une grande partie du travail a donc été réalisée de manière autonome par les différents membres du groupe.

Cela a parfois conduit à une situation où certaines tâches étaient réalisées rapidement à la dernière minute ou concentrées sur quelques personnes.

Nous n'avons pas encore trouvé de solution parfaite pour ce problème d'organisation, mais les difficultés rencontrées nous ont permis de mieux comprendre les contraintes du travail en groupe et d'avancer avec plus de clarté sur la manière dont nous devons communiquer et coordonner notre travail.

Nous cherchons activement des solutions et essayons, avec chaque réalisation d'en tirer une solution.

Difficulté à répartir certaines tâches:

Certaines tâches du projet, en particulier celles liées à la programmation du jeu, sont difficiles à diviser entre plusieurs personnes.

Pour développer un système de jeu fonctionnel, il est souvent nécessaire d'avoir une vision globale du projet et de comprendre comment les différentes parties du code interagissent entre elles. Cette vision globale est parfois difficile à partager entre plusieurs développeurs, surtout lorsque chacun travaille de son côté.

Pour essayer d'améliorer cela, nous avons commencé à mettre en place des chartes et des descriptions plus précises des différentes parties du jeu. Notamment en créant un nouveau canal discord dédié à nos documents ainsi qu'un google drive. L'objectif est de mieux définir les systèmes afin que chacun puisse comprendre plus facilement le travail réalisé par les autres membres du groupe.

Cette réalisation nous a permis de comprendre la nécessité de documenter nos avancées et nos idées au fur et à mesure.

Travail abandonné ou non utilisé:

Au cours du projet, certaines parties du travail réalisé n'ont finalement pas été utilisées dans la version actuelle du jeu.

Par exemple, plusieurs propositions de lore rédigées par Ghali n'ont pas été retenues dans la version finale de l'histoire. Une partie du code réalisé par Julia n'est plus utilisée aujourd'hui et certaines maps créées par Hadrien ont dû être modifiées ou refaites.

Même si ces situations peuvent donner l'impression d'avoir perdu du temps, elles font également partie du processus normal de création d'un projet. Ces étapes nous ont permis de tester différentes idées et de mieux comprendre ce qui fonctionne ou non dans le jeu.

Mauvaise anticipation de la soundtrack:

La création de la soundtrack du jeu n'a pas été réalisée comme prévu initialement.

Au début du projet, nous pensions que la bande sonore pourrait être créée indépendamment du reste du développement. Cependant, nous avons rapidement compris que les sons et les musiques dépendent fortement des actions présentes dans le jeu.

Cette expérience nous a montré qu'il est parfois difficile de planifier certaines tâches sans avoir d'abord expérimenté concrètement les autres parties du projet.

Problème dans la planification de l'IA et du réseau:

Un autre problème vient du fait que nous avons initialement regroupé l'intelligence artificielle et le réseau dans une seule tâche.

Nous nous sommes rapidement rendu compte que ces deux domaines sont très différents et nécessitent chacun beaucoup de travail. De plus, une grande partie du fonctionnement du jeu repose sur le système de réseau, notamment pour permettre la coopération entre les joueurs.

Nous avons donc compris qu'il aurait été plus pertinent de séparer ces deux parties du projet dès le début afin de mieux répartir le travail.

Difficulté à atteindre un objectif de 100 % pour l'IA:

L'objectif d'avoir 100 % de l'intelligence artificielle pour la deuxième soutenance s'est également révélé difficile à atteindre.

Contrairement à d'autres éléments du projet, l'IA ne peut pas être développée totalement séparément du reste du jeu. Elle doit être intégrée directement dans les mécaniques et les niveaux du jeu.

Par exemple, une démonstration d'IA a été réalisée par Julia dans une petite simulation. Cependant, ce code devra être largement adapté pour fonctionner avec les véritables salles du jeu.

Cela montre que l'IA dépend fortement de l'avancement global du projet.

Difficulté à finaliser le level design et le game design:

Enfin, nous avons constaté que le level design et le game design évoluent en même temps que le développement du jeu.

Même si certaines salles et mécaniques peuvent être imaginées sur le papier, leur mise en place dans le jeu peut révéler de nouvelles difficultés. Certaines idées peuvent être trop complexes à programmer, tandis que d'autres peuvent finalement être trop simples pour offrir un défi intéressant aux joueurs.

Le level design et le game design doivent donc souvent être ajustés progressivement en fonction de ce qui fonctionne réellement dans le jeu.

Conclusion:

Les difficultés rencontrées lors de la préparation de la deuxième soutenance sont principalement liées à des problèmes d'anticipation, d'organisation et de planification.

Cependant, ces difficultés nous ont permis de mieux comprendre les contraintes du développement d'un jeu vidéo et d'avancer avec une vision plus claire de la manière dont nous devons organiser notre travail pour la suite du projet.

Leçons tirées pour la suite du projet:

Les difficultés rencontrées lors de la préparation de la deuxième soutenance nous ont permis de mieux comprendre les contraintes liées à la gestion d'un projet de développement de jeu vidéo.

Nous avons notamment compris l'importance de définir plus clairement certaines tâches dès le début, afin de pouvoir mieux estimer l'avancement du projet et répartir le travail entre les membres du groupe.

Nous avons également réalisé que certaines parties du projet, comme l'IA, la soundtrack ou le level design, ne peuvent pas toujours être planifiées de manière totalement indépendante, car elles dépendent fortement de l'avancement concret du développement du jeu.

Enfin, cette expérience nous a montré que les phases d'essais, d'erreurs et de modifications font partie du processus normal de création d'un projet. Même si certaines parties du travail doivent être refaites ou modifiées, elles permettent d'améliorer progressivement le projet et d'avancer avec une meilleure compréhension des objectifs à atteindre.

Nouvelles avancées:

Développement de l'IA et de la simulation:

Avancés de Julia:

Pour cette soutenance, j'ai été chargée de développer un système d'intelligence artificielle pour les ennemis du jeu. Ce travail a représenté un vrai défi car je ne savais pas par où commencer. En cherchant des ressources en ligne, j'ai rapidement remarqué que beaucoup de sites restaient vagues sur la manière concrète d'implémenter une IA dans un jeu en Python. J'ai donc demandé à un collègue ce qu'il utilisait dans son propre projet, et c'est lui qui m'a orienté vers l'algorithme A*, en précisant que c'était particulièrement adapté aux maps créées avec Tiled.

À partir de là, j'ai décidé de réaliser une mini-simulation indépendante en Python avec la bibliothèque Pygame plutôt que d'intégrer directement l'IA dans le jeu principal. Ce choix s'est imposé naturellement : le développement de la première salle, qui ne contient pas encore d'IA, était toujours en cours. Plutôt que d'attendre son achèvement pour commencer à travailler sur la deuxième salle, j'ai préféré avancer en parallèle sur une simulation autonome. Cela m'a permis de développer et de tester le comportement des ennemis dans un environnement simple et contrôlé, avec l'objectif d'adapter ce code aux vraies salles du jeu par la suite.

La machine à états — une progression par étapes:

Au départ, ma première version de la simulation utilisait la map Tiled déjà créée par Hadrien. Cependant, j'ai rapidement réalisé que cette map était trop grande et trop complexe pour tester une IA : les chemins étaient difficiles à contrôler et cela ralentissait inutilement le développement. J'ai donc décidé de créer un petit labyrinthe directement dans Pygame, sous forme d'une grille 2D codée en dur. Cette décision m'a permis de contrôler précisément la disposition des murs et des espaces libres, ce qui était essentiel pour tester le pathfinding.

Dans cette première version, il n'y avait qu'une seule IA qui se contentait de poursuivre le joueur et de s'arrêter sur lui au contact. Le comportement était fonctionnel mais très basique : l'ennemi n'avait aucun autre état, il suivait simplement le joueur le plus proche en permanence.

En remarquant que la simulation impliquait deux joueurs, j'ai ensuite ajouté un deuxième ennemi. C'est là que j'ai rencontré un nouveau problème : les deux ennemis ne suivaient qu'un seul joueur à la fois, et la taille de leur zone de détection était trop grande, ce qui rendait le jeu injouable. J'ai dû ajuster ces paramètres mais je n'ai pas encore trouvé une solution totalement satisfaisante — c'est un point que je devrai approfondir pour la prochaine soutenance.

C'est à ce moment-là que Nael a rédigé la charte de comportement de l'IA, qui décrit formellement les différents états possibles d'un ennemi. Cette charte a été un vrai déclic pour moi : elle m'a permis de formaliser ce que je faisais de manière intuitive et de comprendre clairement ce que je devais implémenter. J'ai alors structuré le comportement de chaque ennemi autour de trois états distincts :

- Patrouille : lorsqu'aucun joueur n'est détecté à proximité, l'ennemi se déplace automatiquement entre plusieurs points prédéfinis sur la map. Il marque également une courte pause entre chaque point pour rendre son comportement plus naturel.
- Poursuite : dès qu'un joueur entre dans la zone de détection de l'ennemi, celui-ci quitte sa patrouille et se dirige vers le joueur le plus proche en utilisant A*.
- Fuite : si les points de vie de l'ennemi descendent en dessous d'un certain seuil, il interrompt le combat et retourne vers sa position d'origine.

Ces trois états sont gérés par un système de priorités clair : la fuite est toujours prioritaire sur tout, puis vient la poursuite, et enfin la patrouille par défaut.

L'algorithme A — fonctionnement et implémentation:

Pour que les ennemis puissent se déplacer intelligemment en évitant les obstacles, j'ai implémenté l'algorithme de pathfinding A* (A-star). Cet algorithme calcule le chemin le plus court entre deux points sur une grille en tenant compte des murs. Voici le code que j'ai écrit, expliqué pas à pas :

La fonction heuristique :

(image annexe 2)

Cette fonction calcule une estimation de la distance entre deux cases de la grille. On utilise la distance de Manhattan — c'est-à-dire la somme des différences en x et en y — car nos ennemis ne se déplacent qu'en 4 directions (haut, bas, gauche, droite), sans diagonales.

L'initialisation de A :

(image annexe 3)

- **open_set** : la liste des cases à explorer, triée automatiquement par priorité grâce à heapq. La case avec le score le plus bas est toujours explorée en premier.
- **came_from** : un dictionnaire qui mémorise, pour chaque case par quelle case on y est arrivé. C'est ce qui permet de reconstituer le chemin à la fin.
- **g_score** : le coût réel pour aller du départ jusqu'à chaque case. Au départ, seul le point de départ a un coût de 0.
- **f_score** : le coût estimé total (g_score + heuristique). C'est ce score qui guide A* vers la cible.

La boucle principale:

(image annexe 4)

- **heapq.heappop** : récupère la case avec le f_score le plus bas — celle qui semble la plus prometteuse.
- **if current == goal** : on a atteint la cible ! On remonte alors le chemin en suivant came_from depuis la cible jusqu'au départ, puis on l'inverse avec reverse() pour l'avoir dans le bon sens.

L'exploration des voisins:

(image annexe 5)

- **for dx, dy in [(0,-1),(0,1),(-1,0),(1,0)]** : on explore les 4 cases voisines (haut, bas, gauche, droite).
- **if not (0 <= ny < rows ...)** : on vérifie que le voisin est bien dans les limites de la grille.
- **if grid[ny][nx] == 1** : si c'est un mur, on ignore cette case.
- **tentative_g = g_score[current] + 1** : le coût pour atteindre ce voisin est le coût actuel + 1 (chaque déplacement coûte 1).
- **if tentative_g < g_score.get(neighbor, float('inf'))** : si on a trouvé un chemin moins coûteux pour atteindre ce voisin, on met à jour ses scores et on l'ajoute à open_set pour l'explorer plus tard.

L'optimisation des performances:

Calculer A* à chaque frame serait trop coûteux en ressources. J'ai donc ajouté un timer : le chemin n'est calculé que toutes les 30 frames environ. Cela représente environ 2 fois par seconde à 60 FPS, ce qui est suffisant pour que l'IA reste réactive sans surcharger le processeur.

Les deux types d'ennemis:

Une fois la machine à états et le pathfinding fonctionnels, j'ai voulu rendre la simulation plus intéressante en implémentant deux types d'ennemis distincts, inspirés de la charte rédigée par Nael. L'idée était de montrer que le même système de base peut donner des comportements très différents selon les paramètres et les actions de chaque ennemi.

L'ennemi de mêlée (rectangle rouge) adopte un comportement agressif et direct. Dès qu'il détecte un joueur dans sa zone de détection, il calcule via A* le chemin le plus court pour l'atteindre et fonce vers lui. Lorsqu'il arrive au contact, il repousse le joueur avec un effet de knockback — un recul calculé dans la direction opposée à l'ennemi — et déclenche un flash rouge temporaire sur le joueur touché pour signaler visuellement le coup reçu. Une période d'invincibilité temporaire empêche l'ennemi d'infliger des dégâts en continu à chaque frame.

L'ennemi archer (rectangle violet) adopte un comportement plus tactique. Au lieu de foncer sur le joueur, il cherche à maintenir une distance optimale pour tirer des projectiles visibles qui se déplacent à travers la map. Si le joueur est dans sa portée de tir, l'archer s'arrête et tire. Si le joueur est trop loin, il utilise A* pour se rapprocher. Un système de cooldown entre chaque tir l'empêche de tirer en continu — il doit attendre environ une seconde entre chaque projectile. Les projectiles disparaissent automatiquement s'ils touchent un mur ou après un certain temps.

Ce qui est intéressant avec ces deux ennemis, c'est qu'ils partagent exactement le même système de machine à états et le même algorithme A*, mais leur comportement en combat est totalement différent. Cela montre la flexibilité du système mis en place et donne un aperçu de ce qui pourra être développé dans les vraies salles du jeu.

La simulation finale:

Pour rendre la démonstration concrète et présentable lors de la soutenance, j'ai intégré tous ces éléments dans une simulation jouable en plein écran. La map est un couloir avec des obstacles générés automatiquement en fonction de la résolution de l'écran, ce qui permet à la simulation de s'adapter à n'importe quel ordinateur sans modification du code.

Les deux joueurs partent du côté gauche de la map — le joueur 1 se contrôle avec ZQSD et le joueur 2 avec les flèches directionnelles. L'objectif est d'atteindre la sortie marquée EXIT à droite de la map, tout en évitant les deux ennemis qui patrouillent dans leurs zones respectives et attaquent dès qu'un joueur s'approche. Une fois les deux joueurs sortis, un écran de victoire s'affiche avec la possibilité de rejouer en appuyant sur R. La touche Échap permet de quitter à tout moment.

Pour la suite du projet, ce code devra être adapté pour fonctionner avec les vraies salles du jeu. Plusieurs points restent à améliorer : la gestion de plusieurs ennemis qui se coordonnent entre eux, l'intégration avec le système réseau pour que chaque joueur soit sur un ordinateur différent, et le remplacement des rectangles de couleur par les vrais sprites graphiques créés par Hadrien.

Avancés de Ghali:

En tant que chargé de programmation, j'ai d'abord essayé de compléter la salle tutoriel moi même, pour ce j'ai appris à utiliser le logiciel TILED et actuellement je suis dans le processus d'ajouter les collisions à cette salle.

Voici l'image que j'ai en tête pour la salle tuto:

(image annexe 6)

Pour ce qui est du puzzle de la salle, l'objectif des joueurs sera de trouver les cristaux cachés quelque part.

Actuellement la salle n'a pas encore les collisions des murs ajoutés mais j'ai pu ajouter l'option de jouer à deux, de même j'ai déjà les scripts écrits pour tous les objets interactifs dont des objets qui seront possible à interagir avec mais qui n'auront pas d'utilité à l'objectif principal.

J'ai pour objectif de finir cette salle bientôt après avoir fini mon système d' IA.

Deuxièmement, je me suis chargé de la deuxième IA qu'on allait ajouter. J'ai souhaité explorer un système de combat de tour à tour. Mon but a été de coder ce système en question. En ce qui concerne le code j'ai implémenté deux classes soit une pour les ennemis et une pour les joueurs.

En ce qui concerne l'ennemi, celui-ci a plusieurs types et chaque type a ses propres attaques et ses propres dégâts.

De même l'ennemi à l'option d'appeler des renforts si il est à base vie, sachant qu' il n'a que 20% de chance de réussite et que si des renforts arrivent l'ennemi meurt en résultat.

Mon futur objectif est d'implémenter les sprites et les animations pour ajouter une interface plus propre et fonctionnelle.

Pour ce qui est des actions faite par mon IA j'ai directement demandé a Claude.Ai de me dicter les différentes actions que l'ia devrait faire selon ses points de vie , les points de vie du joueur, etc.

Site internet :

Avancés de Mélissa :

Pour cette deuxième soutenance, nous avons entièrement refait le site du projet (voir *annexe 1*).

La première version était fonctionnelle mais très basique : une barre de navigation simple, du texte brut sur fond blanc, et une page d'accueil avec juste un bouton "Découvrir" posé sur une image de fond. Le style était minimal, sans identité visuelle particulière.

Pour la nouvelle version, on a voulu que le site reflète vraiment l'ambiance du jeu. On a créé une identité visuelle cohérente avec une palette sombre et des accents verts, qui rappelle l'univers du cristal. La page d'index a été complètement repensée avec un effet de scan animé, des coins lumineux, et un titre avec effet glitch.

On a aussi ajouté plusieurs effets interactifs : un curseur personnalisé qui réagit aux éléments cliquables, un canvas d'arrière-plan avec des particules et une grille d'hexagones animés, des transitions fluides entre les pages, et un effet typewriter sur le sous-titre.

Le contenu a également été restructuré. La page d'accueil présente maintenant le projet avec des statistiques mises en avant et des panneaux organisés, plutôt qu'un simple bloc de texte.

En résumé, on est passés d'un site informatif basique à un site qui a une vraie identité visuelle et qui donne envie de découvrir le projet.

Réseaux :

Avancés de Hadrien :

Dans le cadre du projet SAE J3D "What's Next?" à EPITA, le développement de la composante réseau a constitué une étape stratégique pour transformer un prototype

solo fonctionnel en application multijoueur coopérative respectant scrupuleusement les spécifications du cahier des charges. L'objectif principal consistait à préserver intégralement le code existant de mon coéquipier - incluant le moteur de rendu Pygame, la classe Player avec animation sprite 4 directions, et la classe Room basée sur PyTMX pour le chargement des cartes Tiled - tout en ajoutant une couche réseau transparente assurant une synchronisation parfaite des deux joueurs.

La première phase du développement a porté sur la conception et l'implémentation du serveur TCP autoritaire. Ce serveur écoute sur l'adresse `0.0.0.0:5555`, permettant l'accès depuis toutes les interfaces réseau de la machine hôte (WiFi, Ethernet). L'option socket `SO_REUSEADDR` a été activée dès le départ pour contourner les délais d'indisponibilité du port lors des redémarrages fréquents en phase de test. Chaque client connecté bénéficie d'un thread de gestion dédié via `threading.Thread(target=handle_client)`, garantissant un traitement non-bloquant et simultané des données des deux joueurs.

Au centre de l'architecture se trouve la structure de données `game_state`, implémentée comme un dictionnaire Python centralisé constituant la source unique de vérité pour l'ensemble du système. Cette structure stocke les positions absolues des deux joueurs (`player1: {"x": 300, "y": 300}`), leur direction d'animation, l'identifiant de la salle courante, ainsi que l'état des mécanismes de jeu (puzzles statues, interrupteurs, objets collectibles). Lorsqu'un client transmet ses inputs sous forme de JSON compact (`{"x": 310, "y": 295, "direction": "right"}`), le thread correspondant met à jour l'état officiel puis déclenche un broadcast immédiat vers tous les clients connectés. Cette approche "authoritative server" élimine tout risque de désynchronisation et constitue la norme industrielle pour les jeux multijoueurs.

L'intégration côté client a requis une refonte minutieuse de la boucle principale Pygame afin de maintenir les 60 images par seconde tout en intégrant la communication bidirectionnelle. La réception des mises à jour serveur s'effectue dans un thread daemon asynchrone dont la logique repose sur un buffer circulaire avec délimiteur `\n`. Cette technique résout élégamment les problèmes de fragmentation TCP où un message JSON peut être reçu en plusieurs paquets discontinus : le buffer accumule les données brutes jusqu'à reconstitution du message complet, puis effectue le parsing via `json.loads()`.

La transmission des inputs joueur implémente un throttling temporel à 50 millisecondes (20 FPS réseau) via `pygame.time.get_ticks()`, réduisant la charge réseau de 66% par rapport à une synchronisation à 60 FPS sans impact perceptible sur la réactivité. La classe Player existante a été étendue d'une unique méthode `get_input(pygame.key.get_pressed())` qui convertit les contrôles ZQSD en dictionnaire JSON prêt à l'emploi, tandis que la méthode de rendu `draw()` utilise désormais exclusivement les coordonnées officielles extraites de `network.game_state['player1']`. La classe Room reste totalement inchangée,

assurant une parfaite continuité avec les assets Tiled développés par l'équipe graphique.

Le processus de validation a révélé plusieurs obstacles techniques majeurs, tous résolus au fil des itérations. Initialement, le serveur se terminait après l'acceptation des deux clients, provoquant des exceptions `BrokenPipeError` lors des tentatives d'envoi suivantes. Cette anomalie a été corrigée par l'ajout d'une boucle infinie de maintien d'activité (`while True: pass`) après la phase d'initialisation des connexions. Par ailleurs, l'utilisation simultanée de plusieurs fenêtres Pygame générait des plantages systématiques lors du changement de focus, dus à la propagation croisée des événements `pygame.QUIT`. Une gestion individualisée des événements par fenêtre, combinée à une fermeture propre des sockets (`sock.close()` dans bloc `finally`), a résolu cette instabilité.

Un autre problème critique concernait le spam réseau excessif générant 3000 messages par seconde, saturant la console et la bande passante. Le throttling à 20 FPS, associé à une gestion silencieuse des exceptions dans `network.py`, a ramené la charge à un niveau optimal. Enfin, les processus zombies persistants sur le port 5555 après fermeture brutale des terminaux ont nécessité la mise en place d'une procédure standard de nettoyage : `kill -9 $(lsof -t -i :5555)` exécutée systématiquement avant chaque relance.

Les tests finaux, réalisés en configuration multi-fenêtres côte-à-côte, ont validé une synchronisation parfaite : les déplacements effectués via ZQSD sur une instance se répercutent instantanément sur l'autre avec une latence locale inférieure à 3 millisecondes. Le système démontre une robustesse exemplaire face aux changements de focus, fermetures intempestives et reconnexions. La transition vers une configuration LAN s'avère triviale via substitution de l'adresse IP (`ifconfig | grep inet` sur la machine hôte).

L'architecture développée offre une scalabilité native pour les 7 à 8 salles prévues au cahier des charges : l'ajout d'une salle nécessite simplement l'extension de `game_state['current_room']`, tandis que les mécanismes de puzzles (séquence statues Racine-Soleil-Lune, interrupteurs de ponts) s'intègrent via validation serveur dans `handle_client()` avant broadcast. La collecte d'objets existante dans `Room.check_items()` sera appelée côté serveur sur les positions officielles pour garantir la cohérence.

Ce système réseau, représentant environ 800 lignes de code Python3 optimisé, permet à What's Next? de satisfaire pleinement les exigences multijoueurs de la formation EPITA tout en préservant l'intégralité du travail graphique et de gameplay préexistant. La démonstration technique est prête pour évaluation.

Code et salles:

Avancés Nael:

Création des maps dans Tiled:

La première étape a consisté à concevoir les deux salles du donjon directement dans Tiled Map Editor. Chaque map est organisée en plusieurs calques de tuiles superposés :

- Calque sol
- dalles de fond de la salle
- Calque wall / wall tuiles
- murs et obstacles solides
- Calque deco
- éléments décoratifs (torches, statues, coffres...)
- Calque d'objets
- objets interactifs placés manuellement (items, portes, spawns)

Le tileset utilisé est `DungeonCrawl_ProjectUtumnoTileset.png`, un tileset open-source de type donjon. Il est référencé dans un fichier `donjon 1.tsx` partagé entre les deux maps.

Configuration du tileset et résolution des chemins:

Un problème de chemin relatif est apparu lors du transfert des fichiers d'un PC à l'autre : le fichier `.tmx` référençait le tileset via un chemin absolu lié au PC d'origine. La solution a été de corriger manuellement le chemin source dans le fichier `.tsx` pour pointer vers le dossier `assets/tilesets/` du projet.

Chargement de la map dans Pygame avec `pytmx`:

La classe `Room` a été développée pour charger et afficher les maps Tiled dans Pygame. Elle utilise `pytmx.load_pygame()` pour parser le fichier `.tmx`, puis itère sur les calques de tuiles visibles pour les afficher à l'écran tuile par tuile avec `surface.blit()`.

Création du joueur et gestion des sprites:

Deux personnages jouables ont été intégrés, chacun avec son propre spritesheet (4 colonnes × 3 lignes, 500×500 px) :

- Joueur 1 (elfe bleu) — contrôlé aux touches directionnelles
- Joueur 2 (elfe vert) — contrôlé aux touches ZQSD

Le personnage est extrait du spritesheet via `subsurface()` et redimensionné à 48×64 px. Le déplacement gauche/droite utilise `pygame.transform.flip()` pour éviter d'avoir besoin d'une ligne dédiée dans le spritesheet. L'animation est gérée par un compteur de frame incrémenté à chaque déplacement.

Système de collisions avec les murs:

Les collisions sont générées automatiquement depuis le calque wall tuiles de Tiled. Chaque tuile non nulle de ce calque est convertie en un `pygame.Rect`. Lors du déplacement, si le rect du joueur entre en collision avec un mur, la position est annulée (rollback sur `old_x`, `old_y`).

Objets collectibles:

Un système d'items collectibles a été mis en place via le calque d'objets de Tiled. Chaque objet avec la classe collectible est chargé dans une liste. Lors de la collision avec le joueur, l'item devient invisible et un message de notification s'affiche à l'écran pendant 3 secondes dans un encadré centré.

Système de portes et transition entre salles:

Une porte de sortie (classe porte dans Tiled) permet de passer de la salle 1 à la salle 2. La logique de transition impose que les deux joueurs soient simultanément sur la porte pour déclencher le changement de salle. Si un seul joueur est dessus, un message d'avertissement s'affiche : « Les deux joueurs doivent atteindre la porte de sortie ! ». Lors de la transition, les deux joueurs sont repositionnés à leur point de spawn dans la nouvelle salle.

Versionnement avec Git:

Le projet a été porté sur GitHub tout au long du développement. Cela a permis de revenir à des commits stables lors de bugs critiques (fichier `.tmx` corrompu, calque wall tuiles cassé) via la commande `git checkout <id_commit> -- <fichier>`.

Bilan et fonctionnalités implémentées:

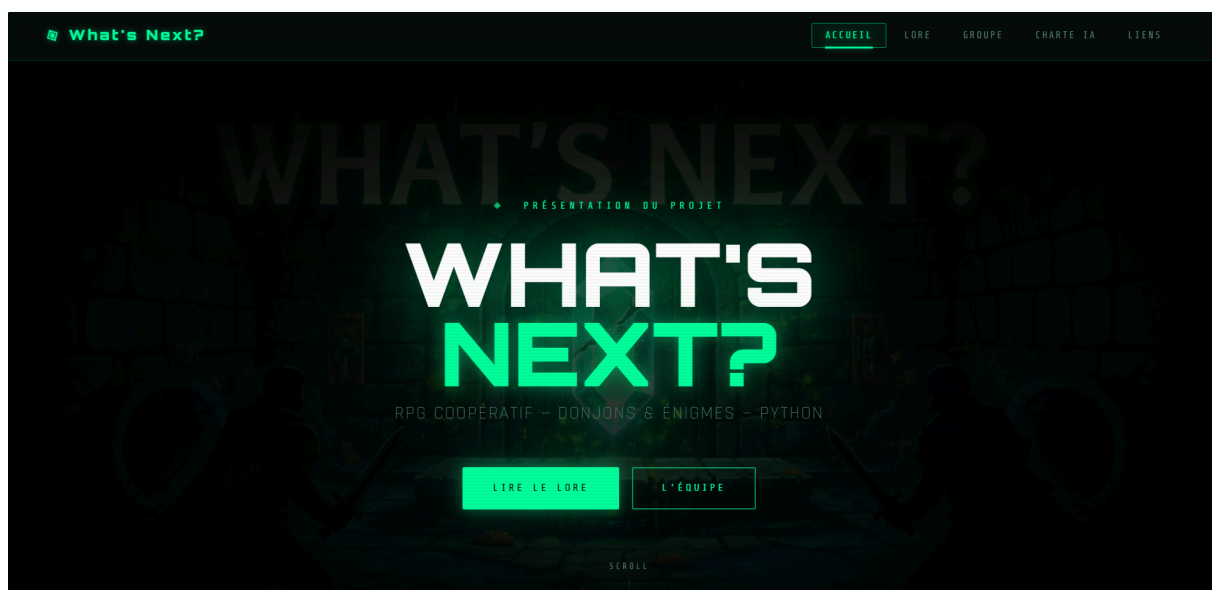
- Affichage de maps Tiled multi-calques dans Pygame
- Deux joueurs contrôlables simultanément en local
- Animation des personnages par défilement de spritesheet
- Collisions solides avec les murs issus de Tiled
- Système d'items collectibles avec notification visuelle
- Portes de transition entre deux salles avec condition coopérative
- Notifications à l'écran avec encadré centré et durée temporisée
- Gestion de projet Git avec branches et historique de commits

Perspectives d'évolution:

- Ajout d'un mode réseau (coop en LAN) avec synchronisation des positions via sockets
- Ennemis avec déplacement et pathfinding
- Système d'inventaire et d'effets des items
- Animations des objets de la map (torches, portes animées)
- Caméra suivant le joueur pour les grandes maps
- Menu principal et écran de fin

Annexe :

1)



What's Next?

ACCUEILLOREGROUPECHARTE IALIENS

HISTOIRE

Le Cristal des Profondeurs

Chroniques souterraines, Chapitre 1



01

Dans le paisible village de ..., deux amis passent leurs journées à aider les habitants et à explorer les environs. Rien ne laissait présager que cette journée serait **différente des autres**.

02

Alors qu'ils traversent la place du marché, une vieille dame les interpelle. Elle semble inquiète : un objet précieux, une **ancienne relique**, a disparu non loin du village. Elle leur promet une récompense s'ils acceptent d'aller la récupérer.

What's Next?

ACCUEILLOR**GROUPE**CHARTE IALIENS

L'EQUIPE

Les créateurs de What's Next?

Notre projet repose sur un travail collaboratif entre plusieurs membres aux compétences complémentaires. Chacun joue un rôle essentiel dans la conception, le développement et l'organisation du jeu.



PROGRAMMATION

Ghali



LEVEL DESIGN

Melissa



GAME DESIGN

Nael

What's Next?

ACCUEILLORGROUPE**CHARTE IA**LIENS

INTELLIGENCE ARTIFICIELLE

Charte de comportement des mobs de donjon

01 - OBJECTIF

Cette IA contrôle un ennemi simple capable de patrouiller, détecter un joueur dans son champ de vision, s'approcher pour attaquer selon son type de combat, et se replier quand sa santé devient trop faible. L'objectif est d'obtenir un comportement lisible, crédible et facile à implémenter.

02 - ETATS PRINCIPAUX

01

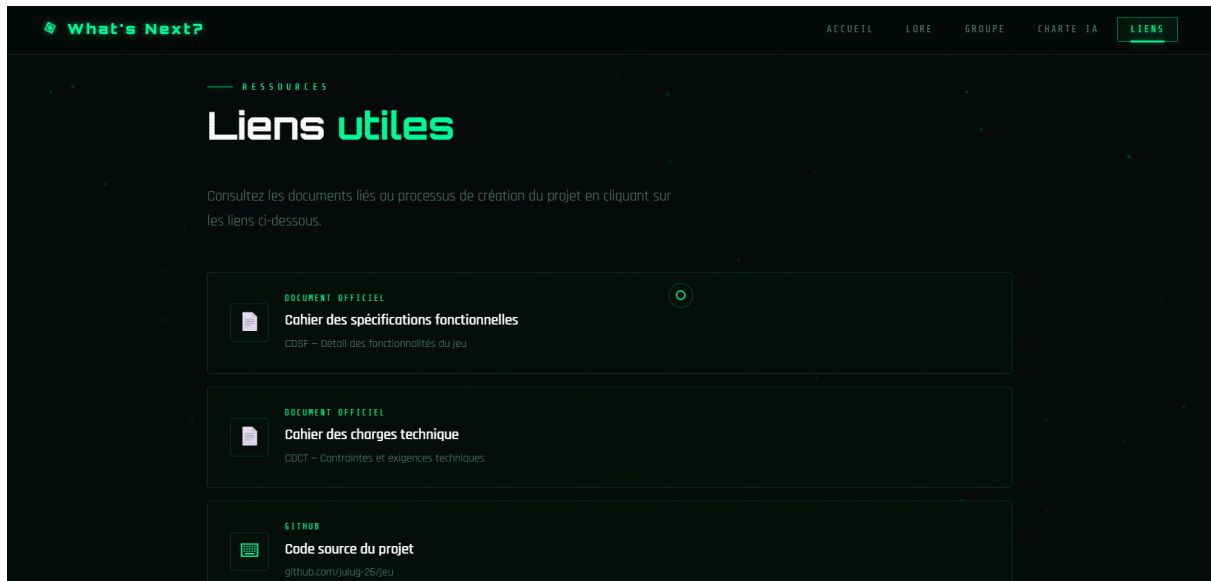
PATROUILLE

Le mob se déplace entre plusieurs points prédéfinis ou marche aléatoirement dans une zone. Il peut marquer une courte pause pour paraître naturel. Dès qu'un joueur est visible, il passe en mode poursuite.

02

DÉTECTION / POURSUITE

Quand un joueur est vu, le mob vérifie la distance, l'angle de vision et l'absence d'obstacle. Il se tourne vers sa cible et avance jusqu'à atteindre sa distance d'attaque idéale.



2)

```
def heuristic(a, b):  
    return abs(a[0] - b[0]) + abs(a[1] - b[1])
```

3)

```
open_set = []  
heapq.heappush(open_set, (0, start))  
  
came_from = {}  
g_score = {start: 0}  
f_score = {start: heuristic(start, goal)}
```

4)

```

while open_set:
    _, current = heapq.heappop(open_set)

    if current == goal:
        path = []
        while current in came_from:
            path.append(current)
            current = came_from[current]
        path.reverse()
        return path

```

5)

```

for dx, dy in [(0,-1),(0,1),(-1,0),(1,0)]:
    nx, ny = current[0] + dx, current[1] + dy

    if not (0 <= ny < rows and 0 <= nx < cols):
        continue
    if grid[ny][nx] == 1:
        continue

    tentative_g = g_score[current] + 1
    neighbor = (nx, ny)

    if tentative_g < g_score.get(neighbor, float('inf')):
        came_from[neighbor] = current
        g_score[neighbor] = tentative_g
        f_score[neighbor] = tentative_g + heuristic(neighbor, goal)
        heapq.heappush(open_set, (f_score[neighbor], neighbor))

```

6)

